Comprehensive Code Intelligence Benchmarking for Zen

Technical Report v2025.09

Zen LM Research Team
research@zenlm.org

September 2025

Abstract

We present a comprehensive evaluation of Zen models on code intelligence tasks, spanning function-level synthesis (HumanEval, MBPP), repository-level engineering (SWE-bench), cross-file completion (RepoBench), code review, test generation, and security vulnerability scanning. Zen-32B achieves 87.4% on HumanEval (pass@1), 81.2% on MBPP, 34.8% on SWE-bench Verified, and 79.3% on RepoBench. We provide detailed analysis by programming language, task type, and repository size, and introduce a real-world coding agent evaluation protocol that extends beyond function-level benchmarks. Security analysis shows 68.4% detection rate on known CWE vulnerabilities.

1 Introduction

Code intelligence evaluation has evolved from simple function synthesis to repository-level tasks requiring understanding of project structure, dependencies, and conventions. The Zen code benchmarking suite covers this full spectrum:

- **Function-level**: Generate functions from docstrings (HumanEval, MBPP)
- **Repository-level**: Resolve GitHub issues requiring multi-file edits (SWE-bench)
- **Completion**: Cross-file code completion given repository context (RepoBench)
- **Code review**: Identify bugs and suggest improvements
- **Test generation**: Write unit tests achieving high coverage
- **Security**: Detect and explain vulnerabilities (CWE taxonomy)

Each dimension tests different capabilities. Function synthesis tests local reasoning; repository-level tasks test project understanding and code navigation.

2 Evaluation Infrastructure

2.1 Execution Environment

All code evaluations use sandboxed Docker containers with:

- Language runtimes: Python 3.11, Node.js 20, Go 1.22, Rust 1.78, Java 21
- Timeout: 10 seconds per test case, 5 minutes per problem
- No network access (prevents web API shortcuts)
- Deterministic test seeds for reproducibility

2.2 Pass@k Estimation

Following Chen et al. [1], we use the unbiased pass@k estimator:

$$\text{pass@}k = \mathbb{E}_{\text{problems}}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right] \tag{1}$$

where n is the number of samples per problem and c is the number that pass all tests. We use $n = 200$ samples for pass@1 estimation and $n = 20$ for pass@10.

3 Function-Level Benchmarks

3.1 HumanEval

Model	pass@1	pass@10	pass@100
Zen-7B	78.4%	91.2%	97.8%
Zen-32B	**87.4%**	**95.1%**	**98.9%**

Table 1: HumanEval results. 164 Python function synthesis problems.

3.1.1 Error Analysis on HumanEval

Among Zen-7B failures (21.6% of problems at pass@1):

Failure Mode	Fraction
Edge case handling (empty input, None, overflow)	38%
Off-by-one errors	22%
Wrong algorithm (correct on examples, wrong in general)	18%
Syntax/runtime error	12%
Incomplete implementation	10%

Table 2: HumanEval failure mode distribution (Zen-7B, pass@1 failures).

3.2 MBPP

MBPP (Mostly Basic Programming Problems) contains 500 Python problems with an average of 3 test cases per problem:

Model	pass@1	pass@3	Difficulty Level
Zen-7B	74.8%	82.4%	Easy: 94.1%, Hard: 52.3%
Zen-32B	**81.2%**	**88.7%**	Easy: 97.2%, Hard: 61.8%

Table 3: MBPP results split by difficulty.

4 Repository-Level: SWE-bench

4.1 Task Definition

SWE-bench [2] presents real GitHub issues from popular Python repositories. The model must generate a patch (unified diff) that resolves the issue and passes the associated test suite. SWE-bench Verified contains 500 carefully validated issue-patch pairs.

4.2 Agentic Evaluation Protocol

We evaluate Zen in an agentic setup where the model can:

1. Read repository files
2. Search for relevant code patterns
3. Run tests to verify hypotheses
4. Apply patch edits iteratively

The agent is given a 30-step budget and access to repository via a file system tool.

Algorithm 1 SWE-bench Agentic Resolution

Require: Issue I, repository R, step budget $B = 30$
Ensure: Patch P

```
Read issue I and relevant files from R
for step = 1 ... B do
    a ← π(I, history) {Generate next action}
    if a is ReadFile: read specified file
    if a is SearchCode: run ripgrep on R
    if a is RunTests: execute test suite, observe output
    if a is EditFile: apply diff to R
    if a is Submit: generate patch P, break
end for
return P
```

4.3 Results

Model	Resolved (%)	Avg. Steps	Token Budget
Zen-7B (agentic)	28.4%	18.2	32K
Zen-32B (agentic)	**34.8%**	21.4	64K
Zen-7B (non-agentic)	12.1%	1	16K

Table 4: SWE-bench Verified results. Agentic setup substantially outperforms one-shot.

4.3.1 Performance by Repository

Repository	Issues	Resolved (Zen-32B)
django	106	38.7%
scikit-learn	72	36.1%
matplotlib	52	28.8%
pytest	40	42.5%
sympy	96	31.2%
requests	30	40.0%
Flask	21	38.1%
Others	83	27.7%

Table 5: SWE-bench Verified resolution rate by repository (Zen-32B).

5 Cross-File Completion: RepoBench

5.1 Task Definition

RepoBench evaluates code completion that requires retrieving and using context from other files in the same repository. Each problem specifies a file, a cursor position, and a ground-truth completion token that requires cross-file context.

Model	Python	Java	TypeScript	Go	Avg.
Zen-7B	76.4%	74.2%	72.8%	78.1%	75.4%
Zen-32B	**81.8%**	**79.4%**	**77.3%**	**82.6%**	**80.3%**

Table 6: RepoBench exact match accuracy by language.

6 Language-Specific Performance

We evaluate HumanEval-style benchmarks in multiple programming languages:

Language	Zen-7B pass@1	Zen-32B pass@1	Benchmark	Problems
Python	78.4%	87.4%	HumanEval	164
JavaScript	74.1%	83.8%	HumanEval-JS	164
TypeScript	72.3%	82.1%	MultiPL-E	164
Rust	68.4%	78.3%	MultiPL-E	164
Go	71.2%	80.9%	MultiPL-E	164
Java	73.8%	83.2%	MultiPL-E	164
C++	75.1%	84.3%	MultiPL-E	164
Average	73.3%	82.9%	–	–

Table 7: Cross-language code synthesis performance.

Rust and Go show lower performance due to stricter memory safety requirements and more complex type systems. Performance correlates with training data volume per language.

7 Test Generation

We evaluate test generation quality by:

1. Giving the model a function signature and implementation
2. Asking the model to write unit tests
3. Measuring line coverage and mutation score

Model	Line Coverage	Branch Coverage	Mutation Score	Tests/Function
Zen-7B	78.3%	71.4%	62.1%	4.8
Zen-32B	**84.1%**	**77.8%**	**68.4%**	6.2

Table 8: Test generation quality metrics on 500 Python functions.

8 Security Vulnerability Detection

8.1 Benchmark Design

We construct a security benchmark of 500 code snippets with known CWE vulnerabilities, drawn from CVE database examples and deliberately introduced vulnerabilities:

CWE Category	Samples	Zen-32B Detection
CWE-89: SQL Injection	80	91.2%
CWE-79: XSS	60	88.3%
CWE-78: OS Command Injection	50	84.0%
CWE-22: Path Traversal	50	82.0%
CWE-125: Out-of-bounds Read	60	61.7%
CWE-476: NULL Pointer Deref	40	57.5%
CWE-416: Use After Free	40	52.5%
CWE-190: Integer Overflow	60	48.3%
CWE-798: Hardcoded Credentials	60	83.3%
Overall	500	**68.4%**

Table 9: Security vulnerability detection by CWE category.

Zen models perform well on injection vulnerabilities (SQL, XSS, command injection) which are pattern-recognizable from training data. Memory safety issues (use-after-free, integer overflow) in C/C++ are harder, reflecting the lower volume of C security examples in training data relative to Python web security content.

9 Analysis

9.1 Scaling Laws for Code

Code performance scales with model size following a similar law to natural language:

$$\text{HumanEval}(N) \approx 87.4 - 31.2 \cdot N^{-0.12} \tag{2}$$

where N is the number of active parameters. The exponent (0.12) is slightly larger than for MMLU (0.095), suggesting code tasks benefit more from scale than general knowledge retrieval.

9.2 Context Length and Repository Tasks

Context Budget	RepoBench Accuracy	SWE-bench Resolved
8K	71.2%	18.3%
16K	76.4%	26.1%
32K	79.3%	32.4%
64K	80.1%	34.8%
128K	80.4%	35.2%

Table 10: Impact of context budget on repository-level tasks (Zen-32B).

Repository tasks show strong gains up to 64K context, with diminishing returns beyond. Most repositories contain under 100 relevant files; key context fits within 64K tokens.

10 Conclusion

Zen models achieve strong performance across the code intelligence spectrum, from 87.4% HumanEval pass@1 to 34.8% SWE-bench resolution. The gap between function-level and repository-level performance reflects the additional challenges of project navigation, multi-file reasoning, and test-driven development required for real-world coding tasks. Security vulnerability detection shows the strongest performance on injection-class vulnerabilities; memory safety in low-level languages remains a frontier for improvement.

References

[1] Chen et al. (2021). Evaluating Large Language Models Trained on Code. *arXiv:2107.03374*.

[2] Jimenez et al. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR*.

[3] Liu et al. (2023). RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. *arXiv:2306.03091*.

[4] Austin et al. (2021). Program Synthesis with Large Language Models. *arXiv:2108.07732*.

[5] Cassano et al. (2022). MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation. *arXiv:2208.08227*.